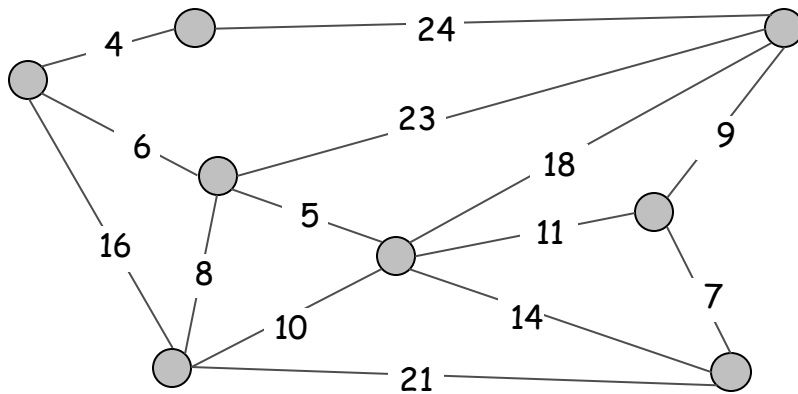


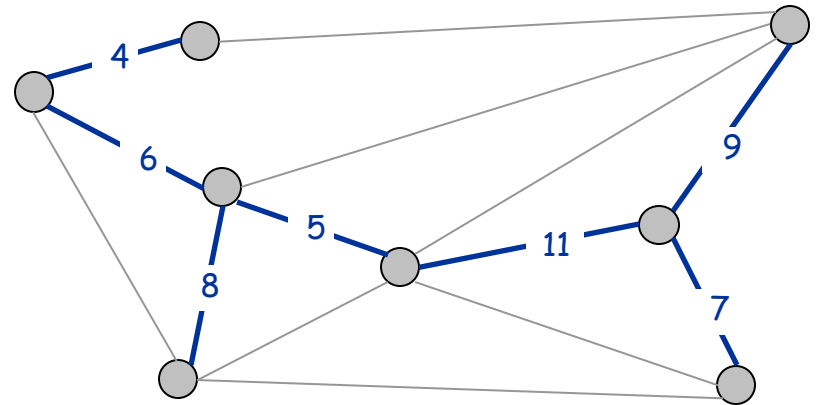
Minimum Spanning Tree

Minimum Spanning Tree

Minimum spanning tree. Given a connected graph $G = (V, E)$ with real-valued edge weights c_e , an MST is a subset of the edges $T \subseteq E$ such that T is a spanning tree whose sum of edge weights is minimized.



$G = (V, E)$



$T, \sum_{e \in T} c_e = 50$

Cayley's Theorem. There are n^{n-2} spanning trees of K_n .

↑
can't solve by brute force

Applications

MST is fundamental problem with diverse applications.

- **Network design.**
 - telephone, electrical, hydraulic, TV cable, computer, road
- Approximation algorithms for NP-hard problems.
 - traveling salesperson problem, Steiner tree
- Indirect applications.
 - max bottleneck paths
 - LDPC codes for error correction
 - image registration with Renyi entropy
 - learning salient features for real-time face verification
 - reducing data storage in sequencing amino acids in a protein
 - model locality of particle interactions in turbulent fluid flows
 - autoconfig protocol for Ethernet bridging to avoid cycles in a network
- **Cluster analysis.**

Greedy Algorithms

Kruskal's algorithm. Start with $T = \emptyset$. Consider edges in ascending order of cost. Insert edge e in T unless doing so would create a cycle.

Reverse-Delete algorithm. Start with $T = E$. Consider edges in descending order of cost. Delete edge e from T unless doing so would disconnect T .

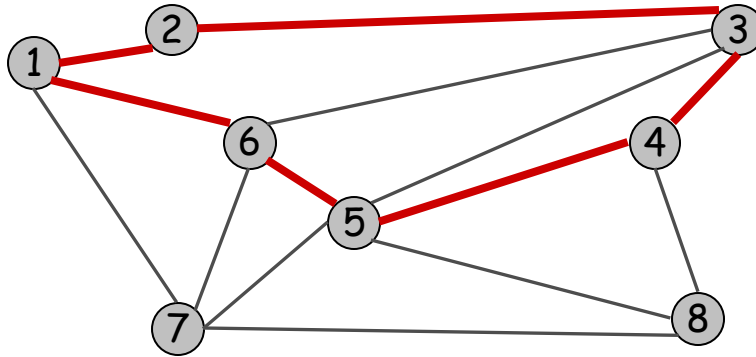
Prim's algorithm. Start with some root node s and greedily grow a tree T from s outward. At each step, add the cheapest edge e to T that has exactly one endpoint in T .

Remark. All three algorithms produce an MST.

DEMO

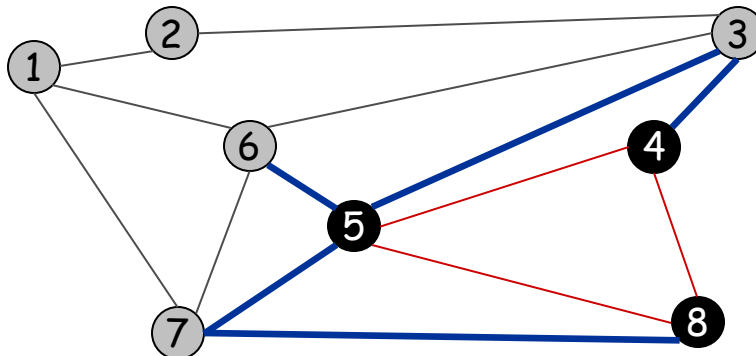
Cycles and Cuts

Cycle. Set of edges the form $a-b, b-c, c-d, \dots, y-z, z-a$.



Cycle $C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1$

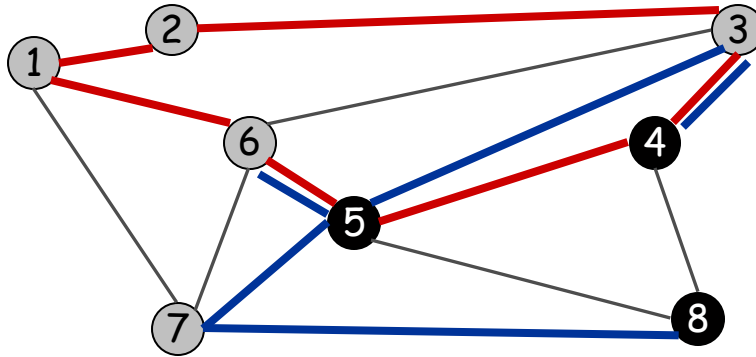
Cutset. A **cut** is a subset of nodes S . The corresponding **cutset** D is the subset of edges with exactly one endpoint in S .



Cut $S = \{4, 5, 8\}$
Cutset $D = 5-6, 5-7, 3-4, 3-5, 7-8$

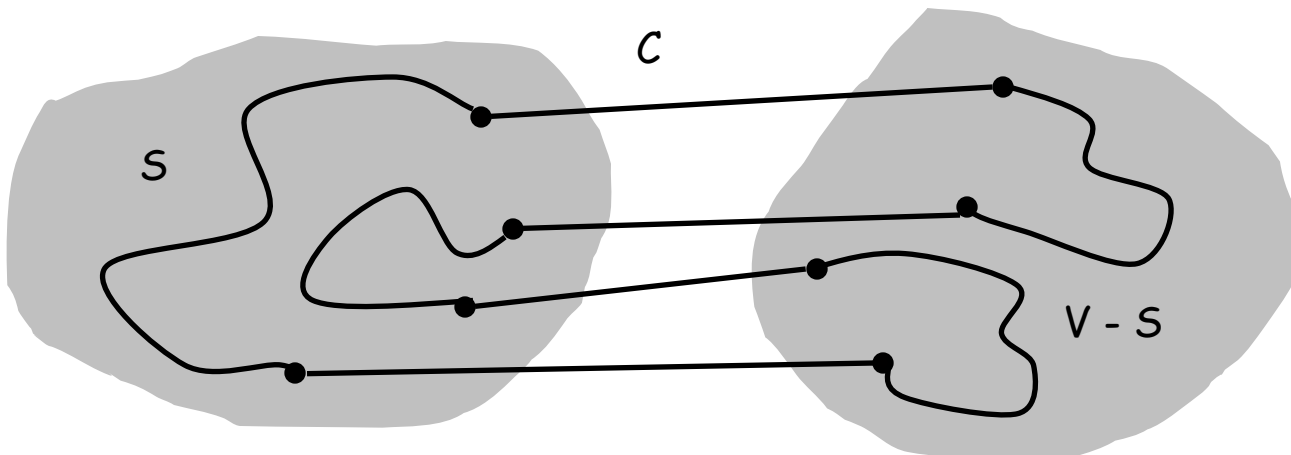
Cycle-Cut Intersection

Claim. A cycle and a cutset intersect in an even number of edges.



Cycle $C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1$
Cutset $D = 3-4, 3-5, 5-6, 5-7, 7-8$
Intersection = $3-4, 5-6$

Pf. (by picture)

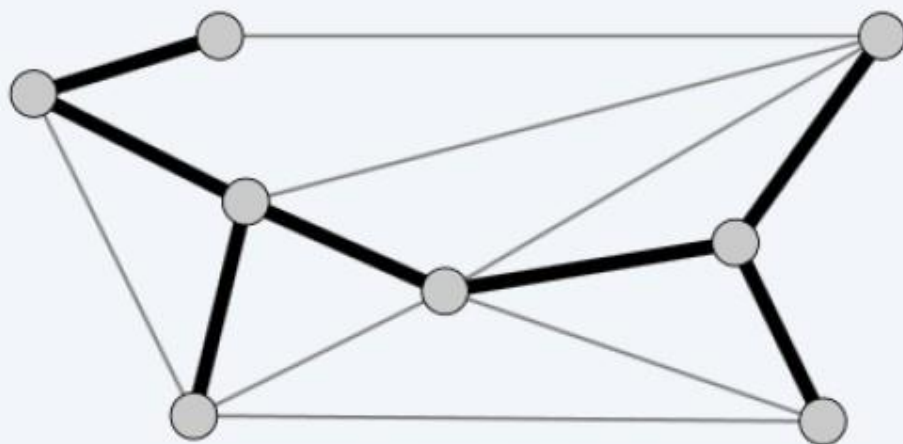


Spanning tree properties

+

Proposition. Let $T = (V, F)$ be a subgraph of $G = (V, E)$. TFAE:

- T is a spanning tree of G .
- T is acyclic and connected.
- T is connected and has $n - 1$ edges.
- T is acyclic and has $n - 1$ edges.
- T is minimally connected: removal of any edge disconnects it.
- T is maximally acyclic: addition of any edge creates a cycle.
- T has a unique simple path between every pair of nodes.



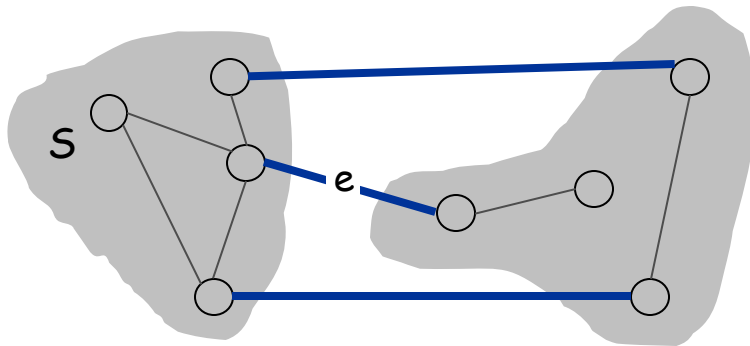
$T = (V, F)$

Greedy Algorithms - Local Properties

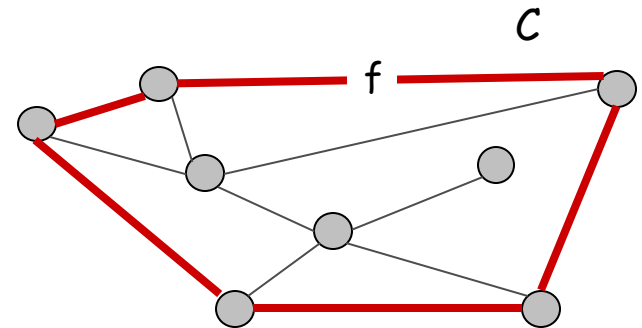
Simplifying assumption. All edge costs c_e are distinct.
(For Determinism)

Cut property. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S . Then the MST contains e .

Cycle property. Let C be any cycle, and let f be the max cost edge belonging to C . Then the MST does not contain f .



e is in the MST



f is not in the MST

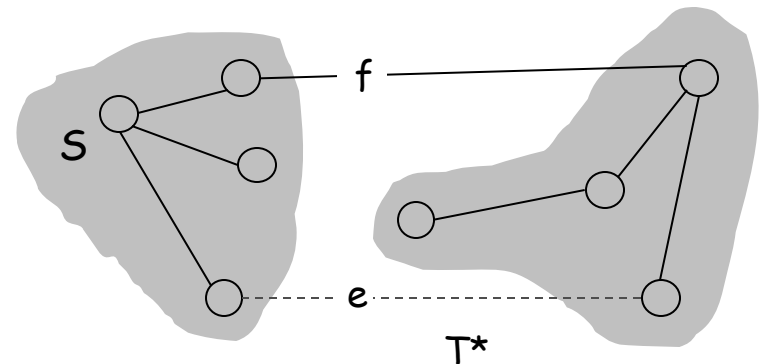
Greedy Algorithms

Simplifying assumption. All edge costs c_e are distinct.

Cut property. Let S be any subset of nodes, and let e be the min cost edge with exactly one endpoint in S . Then the MST T^* contains e .

Pf. (exchange argument)

- Suppose e does not belong to T^* .
- Adding e to T^* creates a cycle C in T^* .
- Edge e is both in the cycle C and in the cutset D corresponding to S
 $\Rightarrow$ there exists another edge, say f , that is in both C and D .
- $T' = T^* \cup \{e\} - \{f\}$ is also a spanning tree.
- Since $c_e < c_f$, $\text{cost}(T') < \text{cost}(T^*)$.
- This is a contradiction. ▀



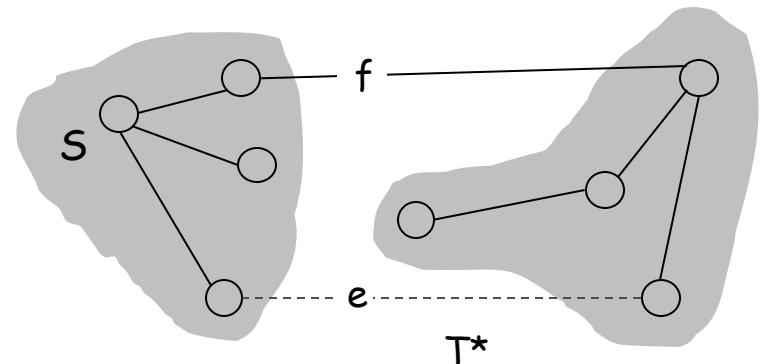
Greedy Algorithms

Simplifying assumption. All edge costs c_e are distinct.

Cycle property. Let C be any cycle in G , and let f be the max cost edge belonging to C . Then the MST T^* does not contain f .

Pf. (exchange argument)

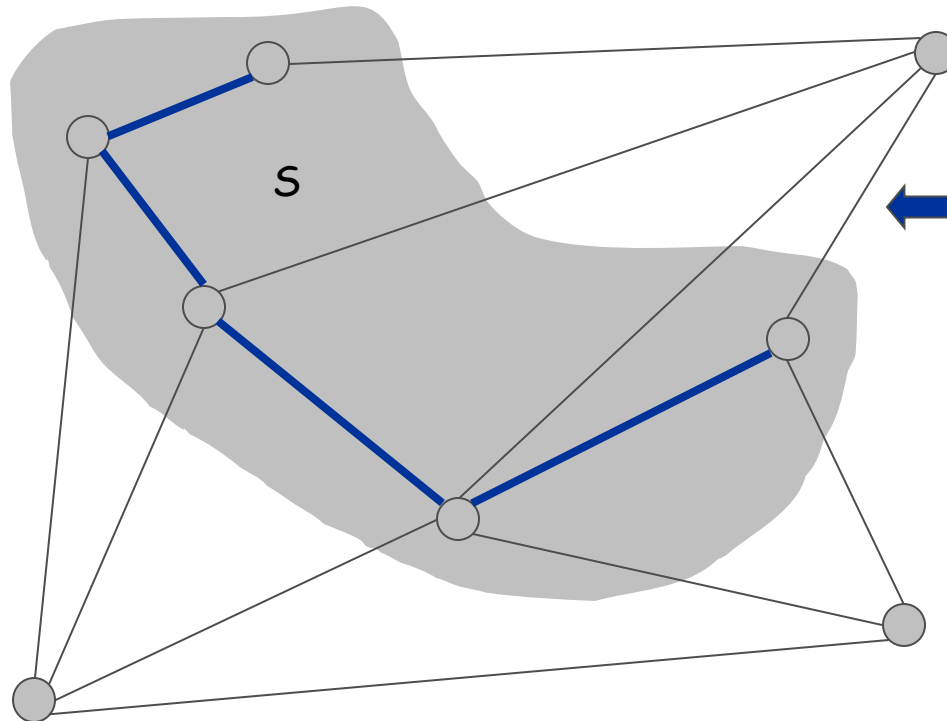
- Suppose f belongs to T^* , and let's see what happens.
- Deleting f from T^* creates a cut S in T^* .
- Edge f is both in the cycle C and in the cutset D corresponding to S
 $\Rightarrow$ there exists another edge, say e , that is in both C and D .
- $T' = T^* \cup \{e\} - \{f\}$ is also a spanning tree.
- **Since $c_e < c_f$, $\text{cost}(T') < \text{cost}(T^*)$.**
- **This is a contradiction. ■**



Prim's Algorithm: Proof of Correctness

Prim's algorithm. [Jarník 1930, Dijkstra 1957, Prim 1959]

- Initialize S = any node.
- Apply **cut property** to S .
- **Add min cost edge in cutset** corresponding to S to T , and add one new explored node u to S .



Implementation: Prim's Algorithm

Implementation. Use a priority queue ala Dijkstra.

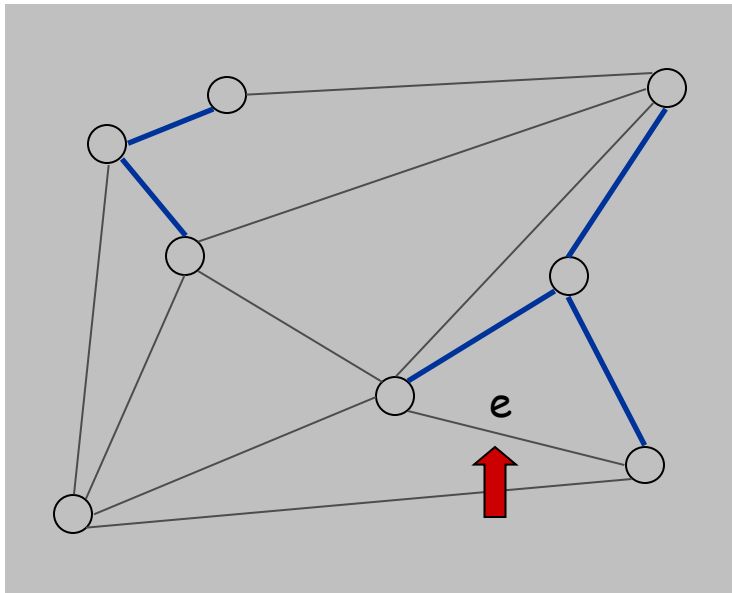
- Maintain set of explored nodes S .
- For each unexplored node v , maintain attachment cost $a[v]$ = cost of cheapest edge v to a node in S .
- $O(n^2)$ with an array; $O(m \log n)$ with a binary heap.

```
Prim(G, c) {  
    foreach (v ∈ V) a[v] ← ∞  
    Initialize an empty priority queue Q  
    foreach (v ∈ V) insert v onto Q  
    Initialize set of explored nodes S ← ∅  
  
    while (Q is not empty) {  
        u ← delete min element from Q  
        S ← S ∪ { u }  
        foreach (edge e = (u, v) incident to u)  
            if ((v ∉ S) and (ce < a[v]))  
                decrease priority a[v] to ce  
    }  
}
```

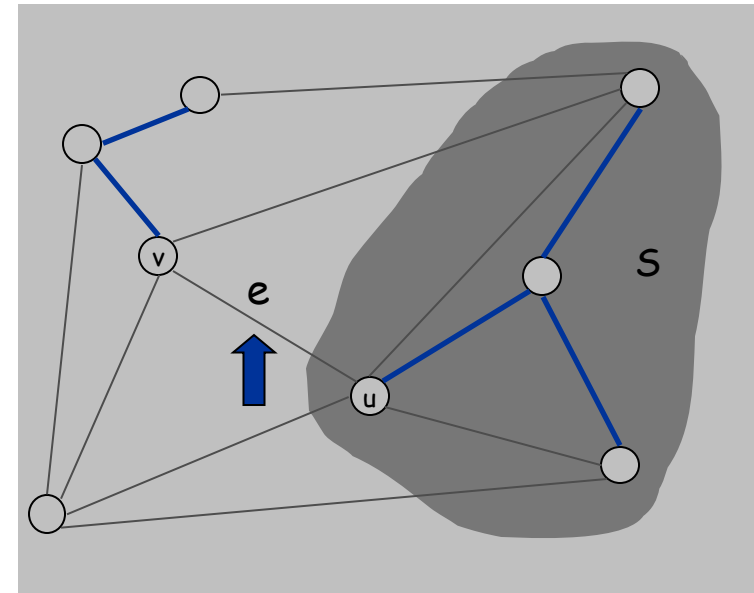
Kruskal's Algorithm: Proof of Correctness

Kruskal's algorithm. [Kruskal, 1956]

- Consider edges in ascending order of weight.
- Case 1: If adding e to T creates a cycle, discard e according to **cycle property**.
- Case 2: Otherwise, insert $e = (u, v)$ into T according to **cut property** where S = set of nodes in u 's connected component.



Case 1



Case 2

Implementation: Kruskal's Algorithm

Implementation. Use the **union-find** data structure.

- Build set T of edges in the MST.
- Maintain set for each connected component.
- $O(m \log n)$ for sorting and $O(m \underbrace{\alpha(m, n)}_{\text{essentially a constant}})$ for union-find.

$\swarrow$ $m \leq n^2 \Rightarrow \log m$ is $O(\log n)$ $\underbrace{\hspace{1.5cm}}$ essentially a constant

```
Kruskal(G, c) {  
    Sort edges weights so that  $c_1 \leq c_2 \leq \dots \leq c_m$ .  
     $T \leftarrow \phi$   
  
    foreach ( $u \in V$ ) make a set containing singleton  $u$   
  
    for  $i = 1$  to  $m$       are  $u$  and  $v$  in different connected components?  
         $(u, v) = e_i$        $\swarrow$   
        if ( $u$  and  $v$  are in different sets) {  
             $T \leftarrow T \cup \{e_i\}$   
            merge the sets containing  $u$  and  $v$   
        }  
         $\swarrow$  merge two components  
    return  $T$   
}
```